

```

#pragma once
#include "Mesh.h"
#include <iostream>
#include <Eigen/Dense>
#include <Eigen/Sparse>

class FEM_Solver
{
public:

    explicit FEM_Solver(const Mesh& m);

    void setMesh(const Mesh& m) { mesh = m;}
    void build_FEM();

    void solve();

    void print_A() { std::cout << "A =\n" <<
Eigen::MatrixXcd(A_) << '\n'; }

private:

    Mesh mesh;

    Eigen::VectorXcd b, u;
    Eigen::SparseMatrix<std::complex<double>> A_;// global
matrix
    std::vector<Eigen::Triplet<std::complex<double>>>
triplets_;

    void handle_triangle(int faceIndex);
    Eigen::Matrix3d local_stiffness(const Eigen::Vector3d& e1,
const Eigen::Vector3d& e2, double area) const;
    Eigen::Matrix3d local_mass(const Vec3& xa, const Vec3& xb,
const Vec3& xc, double area) const;

    double inline triangle_area(const Eigen::Vector3d& e1,
const Eigen::Vector3d& e2) const;
    double potential_on_triangle(const Vec3& xa, const Vec3&
xb, const Vec3& xc) const;

```

```

void build_RHS();

std::complex<double> source_at_point(const Vec3& x) const;
Eigen::Vector3cd local_rhs(const Vec3& xa, const Vec3& xb,
const Vec3& xc, double area) const;

const double hbar = 1;
const double m = 1;

std::complex<double> A0_ = { 1.0, 0.0 };
int ell_ = 1;
double sigma_ = 0.3;

int counter = 1;
};

```